

OWASP Web Security Testing Guide (WSTG) Research and Evaluation

Part I: Framework Notes and Overview

The OWASP Web Security Testing Guide (WSTG) framework outlines a structured methodology across the Software Development Life Cycle (SDLC) to ensure application security controls are actively verified. Testing must be deployed across five core phases of software development:

- **Before Development Begins:** Define a Secure SDLC, review foundational policies and security standards, and establish measurement metrics and traceability criteria to map defects in both process and product.
- **During Definition and Design:** Review security requirements to test underlying assumptions and address structural gaps. Review design and architecture to enforce centralized security controls, and develop Unified Modeling Language (UML) models alongside realistic threat models.
- **During Development:** Conduct comprehensive code walkthroughs with development teams to map logic flows, supplemented by technical code reviews and Static Application Security Testing (SAST) against checklists like the OWASP Top 10.
- **During Deployment:** Execute dynamic application penetration testing (DAST) alongside infrastructure configuration management testing to ensure no default vendor parameters or misconfigurations remain active.
- **During Maintenance and Operations:** Perform regular operational management reviews, periodic health checks, vulnerability regression scans, and strict change verification gates for all patches and system updates.

Part II: Technical Testing Domains and Guidelines

The guide establishes active application analysis split across multiple operational domains, focusing on uncovering technical vulnerabilities, logical flaws, and architectural weaknesses.

1. Authorization Testing

Evaluates if access controls are correctly enforced to prevent privilege manipulation or unauthorized horizontal and vertical access. Core tests include: Path Traversal/Directory Inclusion (WSTG-AZN-01) via input vector enumeration; Bypassing Authorization Schemas (WSTG-AZN-02) via horizontal/vertical traversal; Privilege Escalation (WSTG-AZN-03) via role

manipulation; and Insecure Direct Object References (IDOR) (WSTG-AZN-04) where object parameters are directly altered.

2. Input Validation Testing

Verifies that user-supplied input is securely handled and sanitized before processing. This domain addresses: Reflected and Stored Cross-Site Scripting (XSS), HTTP Parameter Pollution (HPP) to bypass filters, SQL Injection (Inband, Out-of-band, and Inferential/Blind), LDAP/XML/SSI/XPath/IMAP Injection, Code and Command Injections, Format String attacks, Incubated vulnerabilities, and advanced techniques like Server-Side Template Injection (SSTI) and Server-Side Request Forgery (SSRF).

3. Business Logic Testing

Audits custom core workflows and rule definitions to uncover design flaws that automated tools systematically fail to perceive. Focus areas include: Data Validation and request forgery, transaction integrity checks across distributed modules, process timing constraints, function execution limits, circumvention of operational workflows, defenses against general application misuse, and secure file upload controls (unexpected or malicious file handling).

4. Information Gathering and Authentication Testing

Information gathering addresses search engine discovery, server fingerprinting via banner grabbing/malformed requests, metadata file analysis (robots.txt, security.txt), dynamic URL enumeration, source code comment leakage, and application entry path architecture mapping. Authentication testing targets data transit security, default credentials, lockout resilience under high-speed brute-force conditions, authentication schema bypasses, client-side session storage weaknesses (e.g., cleartext or weak hashes), browser cache configuration, password policies, and recovery mechanism patterns.

Part III: Research Section

I. OWASP WSTG Core Definitions and Philosophy

The Open Worldwide Application Security Project (OWASP) is a globally recognized, open-source, non-profit foundation dedicated to improving software robustness. The Web Security Testing Guide (WSTG) is its premier peer-reviewed technical reference architecture, detailing precisely how to audit web applications through systematic manual methodologies combined with automated scanning frameworks. The testing structure operates under a risk-based approach, prioritizing high-impact flaws that pose clear threats to business security

(such as data exfiltration or access control failures) while enforcing a defense-in-depth model that audits every layer of an application stack.

II. Methodology Breakdown: Passive vs. Active Testing

The technical core divides security assessments into two distinct operational phases. Passive testing functions primarily as an observation phase, utilizing reconnaissance techniques to collect target intelligence, map attack surfaces, and identify leaked materials without actively modifying server states. Active testing transitions the assessment into empirical exploitation, requiring direct interaction with the system via input probing, targeted payload execution, and output response analysis across the 12 technical domains of the guide.

III. Empirical Research Case Study: Applying and Evaluating the WSTG

1. Apply OWASP WSTG to Test a Vulnerable Web Application

To practically evaluate whether the framework provides a sufficiently robust testing baseline ("good enough"), the WSTG methodology was applied to a live target environment utilizing the **OWASP Juice Shop** single-page web application. Testing systematically progressed through both passive and active phases to stress the application's implementation controls.

The execution began with **Information Gathering (INFO)**. Client-side compiled JavaScript bundle files (such as main.js) were manually scraped and parsed, uncovering a comprehensive mapping of internal application directory pathways, routing targets, and backend infrastructure components. Network mapping identified crucial architectural API gateways operating under /api and /rest. Passive crawling and parameter fuzzing exposed undocumented peripheral directories, specifically an exposed /ftp repository, alongside a public /metrics scraping endpoint linked to a Prometheus deployment. Infrastructure stack fingerprinting was achieved by purposefully triggering verbose database error handles, which revealed an active SQLite database layer, a Node.js/Express backend framework, and an Angular client-side interface.

Active testing phases then targeted the **Authentication (ATH)** and **Authorization (AZN)** mechanisms. Transport-layer inspection confirmed that credentials were sent over an unencrypted HTTP channel. Application controls were stressed by testing password policy boundaries, brute-force lockout thresholds, security question recovery flows, and session cookie persistence parameters. Finally, advanced **Input Validation (INP)** and **Business Logic (BUS)** test cases were applied to verify the backend's resilience against complex parameter tampering, workflow circumvention, and template manipulation payloads.

2. Identify and Classify Vulnerabilities According to WSTG Testing Categories

The systematic execution of the WSTG test suites successfully exposed a substantial matrix of critical security vulnerabilities. These discovered technical defects have been identified and explicitly classified according to their official WSTG operational domains in the comprehensive taxonomy table below:

WSTG Testing Category	Discovered Vulnerability & Technical Flaw	Practical Impact & Exploitation Mapping
Information Gathering (INFO)	JavaScript Source Comment & Routing Path Exposure	Revealed a comprehensive listing of administrative paths (e.g., /administration, /accounting) directly inside public-facing client scripts.
Information Gathering (INFO)	Infrastructure Leaks via Verbose Errors & Metrics Route	Verbose stack traces fingerprinted SQLite, Angular, and Node.js; the /metrics route exposed live Prometheus performance data.
Information Gathering (INFO)	Exposed Directory via Sensitive Metafiles	A hidden, unlinked /ftp directory allowed full file-system listing and downloading of sensitive internal application files.
Authentication Testing (ATH)	Insecure Plaintext Transport over HTTP Channel	A lack of SSL/TLS allowed active network attackers to sniff login requests, session IDs, and administrative passwords.
Authentication Testing (ATH)	Absence of Lockout Mechanism & Weak Password Policy	The application allowed brief 5-character passwords and omitted rate-limiting controls, enabling high-speed dictionary

WSTG Testing Category	Discovered Vulnerability & Technical Flaw	Practical Impact & Exploitation Mapping
		brute-forcing.
Authentication Testing (ATH)	Insecure Session Token Cryptography & Storage	Browser cookies contained user credentials hashed with obsolete MD5 and encoded in Base64, permitting instantaneous cracking via rainbow tables.
Authentication Testing (ATH)	Predictable Pre-Generated Security Questions	Recovery questions relied on common life events, allowing attackers to completely bypass the authentication schema using OSINT and social engineering.
Authorization Testing (AZN)	Bypassing Authorization Schema via Token Parameters	Altering the bid (Basket ID) numerical parameter inside the session token granted full horizontal access to foreign user shopping carts.
Authorization Testing (AZN)	Vertical & Horizontal Privilege Escalation via API Routes	Executing direct PUT requests to /api and /rest interfaces allowed attackers to tamper with internal records and forge the authorship of product reviews.
Authorization Testing (AZN)	Insecure Direct Object References (IDOR)	Feedback and contact forms trusted client-supplied User ID values within POST payloads, enabling identity theft and request

WSTG Testing Category	Discovered Vulnerability & Technical Flaw	Practical Impact & Exploitation Mapping
		spoofing.
Input Validation Testing (INP)	SQL Injection (SQLi) via Form Fields & Backend APIs	Fuzzing inputs with raw SQL string operators bypassed the login screen entirely (' OR 1=1--), permitting entry into administrative accounts and database schema exfiltration.
Input Validation Testing (INP)	Cross-Site Scripting (DOM-based & Reflected XSS)	The primary search input and product feedback loops directly reflected unfiltered payloads, allowing arbitrary JavaScript execution in victim browsers.
Input Validation Testing (INP)	HTTP Parameter Pollution (HPP) via API PUT Requests	Supplying duplicated query parameter keys in API endpoint calls confused backend parser routing, successfully overriding structural data boundaries.
Input Validation Testing (INP)	Insecure Upload Handling (XML & Null Byte Injection)	File uploads lacked rigorous extension validation; testers used Null Byte strings (.xml\x00.jpg) to bypass sanity checks and inject malicious XML objects.
Input Validation Testing (INP)	Server-Side Template Injection (SSTI) & Code	The username mutation functionality parsed raw input directly into a server template

WSTG Testing Category	Discovered Vulnerability & Technical Flaw	Practical Impact & Exploitation Mapping
	Injection	compiler, resulting in full template injection and remote code execution (RCE).
Business Logic Testing (BUS)	Bypassing Quantity Boundary Data Validation	The cart system failed to enforce positive mathematical limits, enabling users to submit negative numbers (-10) to manipulate pricing and deduct balances.
Business Logic Testing (BUS)	Workflow Circumvention & Coupon Forgery	The promo-code creation mechanism utilized a predictable, reversible client-side algorithm, allowing testers to forge high-value discount vouchers.

3. Compare Manual WSTG-Based Testing with Automated Scanning Tools (OWASP ZAP and Burp Suite)

A primary objective of this thesis is comparing the analytical depth of human intelligence-led manual WSTG testing against fully automated vulnerability scanning software, such as OWASP ZAP and Burp Suite Active Scanner. The empirical findings reveal clear trade-offs in coverage efficiency, false-positive metrics, and contextual application awareness.

Automated scanning utilities demonstrated exceptional efficiency during the initial discovery and mapping phases. Both Burp Suite and OWASP ZAP quickly spidered public web pathways, mapped basic form fields, and flagged structural configuration omissions—such as the total absence of HTTPS encryption. Furthermore, their signature-matching engines successfully detected glaring injection entry points, including the unparameterized SQL Injection flaw inside the standard login interface.

However, automated tools proved entirely ineffective when confronting vulnerabilities that require contextual comprehension or workflow rule verification. Automated scanners treat individual HTTP responses as isolated events; they cannot determine if a structurally valid 200

OK response contains unauthorized data belonging to a different tenant. As a result, both tools completely missed the horizontal authorization bypasses involving the shopping cart bid parameter. Similarly, business logic testing cannot be automated via signature checks: when automated proxy tools fuzzed cart quantity fields with negative numbers, the scanners verified only that the application did not crash (missing the fact that the server processed the negative value to deduct money from the transaction). Automated spiders also struggled to execute dynamic JavaScript event handlers within the Angular framework, causing them to completely overlook nested REST API routes.

In contrast, manual testing structured by the WSTG systematically addresses these limitations by treating security as a thorough investigation of application design assumptions. While manual analysis requires a significantly greater investment of time and human capital, it allows the tester to comprehend complex business interactions, adjust payloads to modern single-page architectures, interpret subtle logic anomalies, and safely uncover advanced flaws like Server-Side Template Injection and code forgery workflows. To summarize these findings, a comparative matrix across core security evaluation metrics is presented below:

Evaluation Parameter	Manual WSTG-Based Testing	Automated Scanning Tools (ZAP / Burp Scanner)
Speed & Operational Efficiency	Extremely time-consuming; requires extensive hours to manually configure payloads and analyze multi-step test inputs.	Extremely rapid; capable of executing thousands of asynchronous requests and signature sweeps across broad scopes in minutes.
Business Logic & Workflow Flaws	High efficacy; human reasoning easily uncovers logical flaws, price manipulation boundaries, and coupon forgery algorithms.	Completely blind; unable to understand structural transaction rules or verify semantic intent.
Access Control & Authorization	Superior accuracy; maps multi-tenant application boundaries to expose horizontal/vertical privilege flaws and IDOR parameters.	Very poor; fails to notice cross-user data leakage if the server returns a standard HTTP 200 message status.

Evaluation Parameter	Manual WSTG-Based Testing	Automated Scanning Tools (ZAP / Burp Scanner)
False Positive / Negative Rates	Minimal false-positive metrics due to precise human verification and empirical proof-of-concept testing.	High false-positive rates on generic code signatures; high false negatives on complex or context-dependent bugs.
Modern Stack Adaptability	Highly adaptable; capable of analyzing asynchronous REST API formats, JSON Web Tokens (JWT), and modern single-page frontends.	Often restricted; automated spiders frequently fail to properly parse dynamic Client-Side scripts or multi-state API conditions.

Part IV: Evaluation and Strategic Improvements

While the OWASP WSTG proves itself to be an exceptional framework ("good enough") for technical vulnerability mapping, its manual execution remains highly slow and operationally complex. To enhance the guide's modern efficacy, organizations should integrate localized automation rulesets for repetitive injection tracking while reserving skilled human capital exclusively for context-rich domains like Access Control and Custom Business Workflows.